

Mltra Conversational Insights - V2 POC Scope

1. Purpose of V2

The V2 POC is intended to improve the existing conversational insights experience and validate whether program teams can naturally use Mltra data through conversation.

The POC has **two equally important purposes**:

1. **Improve the quality of the conversational experience** - especially context retention, relevance, consistency and response structure.
2. **Learn from actual usage** — capture the questions users ask, the types of analysis they seek, and their feedback on the responses.

2. What V2 must be able to do

A. Conversational data exploration

The AI should be able to work across the **entire available Mltra dataset** and answer natural-language questions without requiring users to know the underlying data structure.

Users should be able to ask:

- **Factual questions** — "How many stories were collected?"
- **Comparative questions** — "How does Bihar compare with Karnataka?"
- **Trend questions** — "What changed over the last three months?"
- **Exploratory questions** — "What are the major issues emerging?"
- **Breakdown questions** — "Which districts are contributing most?"
- **Relationship/pattern questions** — "What patterns do we see between X and Y?"
- **Example questions** — "Show me examples of this."
- **Recommendation questions** — "What should the program team focus on?"

B. Core V2 Improvements

1. Context retention

The assistant should remember the context established during the conversation.

For example:

User: How many stories were collected in Bihar in June?

AI: 120 stories...

User: What about Karnataka?

The AI should understand that the user is asking for the **same metric and time period, but a different state**.

Similarly:

User: What about last month?

should change only the relevant time context.

Context to preserve

- State
- District
- Program
- Persona
- Dataset
- Time period
- Metric
- Theme
- Previous result / topic being discussed

2. Response Quality & Structure

1. V2 response principles (based on feedback)

- **1. Answer the user's question first** - Don't make the user read a long introduction before getting the answer.
- **2. Stay focused on the question** - Don't add unrelated analysis simply

- **3. Match response depth to the question** - A simple factual question should receive a short answer. A complex exploratory question can receive a structured analysis.
- **4. Use plain language** - Avoid unnecessary jargon or overly academic/corporate language.
- **5. Don't force every section into every answer** - The response structure should be flexible.

2. Recommended Response Structure

Instead of creating separate rigid templates for every question type, provide the AI with a **recommended structure and guardrails**.

For a detailed insight, the preferred order should be:

- **Table / Key data** - Put the relevant numbers or comparison up front.
- **Executive Summary** - One concise statement explaining the most important finding.
- **Statistical Breakdown** - Where relevant, show the percentage/count distribution, particularly for themes.

For example:

Theme	Reports	%
Theme A	25	42%
Theme B	18	30%
Theme C	10	17%

- **What changed / What is happening** - Explain the actual pattern rather than giving a generic statement.
- **Geographic breakdown** - When geography is relevant, answer: What + Where + When rather than simply listing locations.
- **Interpretation** - Explain what the data suggests, without making unsupported causal claims.
- **Key challenges** → **Key solutions** - If challenges are identified, connect them to relevant actions.
- **Insight card** - Only surface insight cards when they are: relevant to the user's question ; specific to the community/context ; actionable

- "Chart Recommendations" , "Visualization Recommendations" - These should not appear as separate sections in the response.

Add / improve

- Table at the beginning
- Statistical breakdown
- Clearly labelled themes
- Regional leaderboard where the question calls for ranking
- Complete "What Changed" narratives
- Source clarity for "Big Numbers"
- Geographic segmentation with what/where/when
- Challenge → Solution pairing
- Risk level → Theme mapping

3. Data-grounding & Trust

Guardrails.

The AI/ data retrieval system must:

- use the available Mltra datasets as its evidence
- identify the relevant data before answering
- not invent information
- not make unsupported claims
- show counts alongside percentages
- mention the relevant source/dataset where appropriate
- clearly scope findings to the data being analysed
- avoid sweeping conclusions

For example:

✗ "38% of reports mentioned this issue."

✓ "38% (19 of 50 reports) mentioned this issue."

4. Query Capture

Every user query should be captured so that after testing we can understand:

- What are program managers asking?
- What kind of analysis are they expecting?
- Which datasets are they interested in?

What level of analysis do they want?

- State
- District
- Program
- Persona
- Time
- Theme

What questions does the AI struggle with?

Minimum query log

Field	Required
Query ID	Yes
Session ID	Yes
Timestamp	Yes
User query	Yes
AI response	Yes
Feedback	Yes
Feedback reason/comment	Optional
Relevant dataset	If technically available

This query log becomes a major output of the POC.

5. User Feedback

After every response, provide:

Was this response useful?

👍 Yes
👎 No

If the user selects 👎, optionally ask: What could be improved?

V2 Work Breakdown

Conversation

- Persist session/conversation history
- Pass relevant conversation context to the model
- Test multi-turn questions
- Test context switching
- Test follow-up questions
- Prevent unnecessary repetition

AI / Prompt

- Refine V1 prompt
- Add context-preservation instructions
- Add analytical-question guidance
- Add flexible response structure
- Add data-grounding rules
- Add percentage/count rules
- Add concise-response rules
- Add geographic-analysis rules
- Add theme/risk labelling rules

Data

- Make full available Mltra datasets accessible to the AI
- Test factual queries
- Test comparative queries
- Test trends
- Test exploratory analysis
- Test recommendations
- Test cross-dataset questions

Query capture

- Create query/session log

- Capture every user query
- Capture AI response
- Capture timestamp
- Capture session context
- Capture relevant metadata

Feedback

- Add 👍 / 👎
- Store feedback against query ID
- Add optional reason/comment
- Make feedback available for analysis

BUILD PLAN (Suggested)

build plan for the Mltra Conversational Insights Assistant

1. AI Split: Retrieval vs. Interpretation

The spec requires deterministic retrieval, not AI-based retrieval, so that the same question always returns the same numbers. AI is used only for interpretation.

- Retrieval: deterministic Text-to-SQL / fixed query layer against the four source sheets (Bihar MI, Bihar Chaupal, Karnataka MI, Karnataka Chavadi). -Vivek's query_hub can be used for this purpose
- Interpretation (AI/LLM layer only): deciding short-answer vs. Executive Insight format, cause-effect reasoning, theme-tagging judgment calls, generating follow-up questions.
- Redaction happens at ingestion, before data reaches the model, not at generation time.

2. Architecture

Query -> language/entity correction -> session context resolution -> deterministic Text-to-SQL retrieval -> provenance log write -> LLM interpretation layer -> response with citation, rendered back in the dashboard where the question was asked.

3. Real-Time Data Sync (replacing manual entry)

Flow: Bot writes to Sheet -> Apps Script onEdit trigger -> webhook -> redaction/anonymization at ingestion -> upsert to DB -> change_log entry (doubles as audit trace).
One-directional only (sheet to DB) since Mltra never writes back to source sheets.

4. Reference Repos

Text-to-SQL / Retrieval Layer:

- Awesome-LLM-based-Text2SQL (curated index, flags conversational Text-to-SQL benchmarks CoSQL and SParC): <https://github.com/DEEP-PolyU/Awesome-LLM-based-Text2SQL>

- LangChain SQL Q&A tutorial (SQLDatabase + create_sql_query_chain):
https://python.langchain.com/v0.2/docs/tutorials/sql_qa/
- Canner/WrenAI (governed text-to-SQL with semantic layer — reference the semantic-layer pattern, not the full platform): <https://github.com/Canner/WrenAI>
- QueryWeaver (lightweight NL-to-SQL with REST API):
<https://dev.to/githubopensource/queryweaver-ask-your-database-in-plain-english-4cpp>
- gonzalo123/sql_llm (minimal LLM-proposes-SQL, code-executes-deterministically example):
https://github.com/gonzalo123/sql_llm

Provenance / Audit Tracing:

- Text2SqlAgent/text2sql-framework (study [tracing.py/analyze.py](https://github.com/Text2SqlAgent/text2sql-framework) split only; ignore its agentic retrieval approach): <https://github.com/Text2SqlAgent/text2sql-framework>

Real-Time Sync (Sheets to DB):

- Ani-404/google-sheets-mysql-sync (best match — Apps Script trigger, Node.js API, change_log table): <https://github.com/Ani-404/google-sheets-mysql-sync>
- Vimall03/SuperSync (explains event-driven onEdit vs. polling):
<https://github.com/Vimall03/SuperSync>
- Adith-gowda/GoogleSheet-Postgres-Real-Time-Sync (same pattern on Postgres, Flask + ngrok for local dev): <https://github.com/Adith-gowda/GoogleSheet-Postgres-Real-Time-Sync>